AKSUM UNIVERSITY

FACALTY OF COMPUTING TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE

GROUP ASSIGNMENT OF DESIGN AND ANALYSIS OF ALORITHMS

TITLE: GRPH COLORING- EXAM SCHEDULING

PREPARED BY: THRID YEAR COMPUTER SCIENCE STUDENTS

No.	First Name	Last Name	ID Number
1	Haile	Gebremichael	1601860
2	Abel	Tikabo	1603323
3	Lwam	Tesfu	1602205

SUMMITED TO INSTRUCTOR: EKUBAY

SUMMISION DATE: /04/2026GC

Table of Contents

1. Introduction 1

2. Problem Statement 1

3. Algorithm Design 1

4. Why This Algorithm 2

5. Implementation Details 3

6. Experimentation 4

7. Learned Challenges 8

8. Conclusion 9

1. Introduction

Graph colouring is a classical NP-Hard problem in computer science that has significant practical applications in scheduling and resource allocation. Universities face exam scheduling challenges every semester, and Aksum University, with 50 courses across 6 departments, is no exception. Students enrolled in multiple courses create scheduling conflicts, making manual scheduling error-prone and inefficient.

This project models the exam scheduling problem as graph coloring, where vertices represent courses, edges represent student conflicts, and colors represent time slots. The minimum number of colors needed is the chromatic number χ. The goal is to assign time slots so that no student has two exams simultaneously.

Real-world applications of graph coloring include university exam scheduling, compiler register allocation, radio frequency assignment, geographic map coloring, and parallel task scheduling.

2. Problem Statement

Aksum University needs to schedule 50 final exams with the constraint that no student should have two exams in the same time slot. Courses from the same department share many students, creating dense conflicts, while cross-department electives create additional conflicts. Manual scheduling is error-prone, time-consuming, and inefficient.

We need an algorithm that guarantees a conflict-free schedule and finds the MINIMUM number of time slots required, which is equivalent to finding the chromatic number of a graph.

Formal Definition:

- ✓ Given: Graph G = (V, E) where V = 50 courses, E = 228 student-overlap conflicts, m = number of colors (time slots)
- ✓ Find: Minimum m such that for every edge (u,v): color[u] ≠ color[v]
- ✓ This minimum m = chromatic number χ

Graph coloring is NP-Hard, meaning no known polynomial-time algorithm exists.

3. Algorithm Design

Step 1: Build the Graph

- ✓ 50 vertices (courses) and 228 edges (conflicts)
- ✓ Data structure: Adjacency matrix `bool adj[100][100]`
- ✓ Departments: Computer Science (10), Mathematics (10), Physics (8), Electrical Engineering (9), Business (7), Electives (6)

Step 2: Welsh-Powell Greedy Heuristic (Upper Bound)

1. Calculate degree of each vertex
2. Sort vertices by degree in descending order
3. Assign smallest color not used by any neighbor
4. Return maximum color used

✓ Result: Upper bound = 10 colors

Step 3: Backtracking Algorithm (Main Algorithm)

```text
function backtrack(vertex, m):
    if vertex == n: return true
    for color c = 1 to m:
        if isSafe(vertex, c):
            color[vertex] = c
            if backtrack(vertex + 1, m): return true
            color[vertex] = 0
    return false
```

Step 4: Constraint Pruning (isSafe Function)

```text
function isSafe(vertex, c):
    for each neighbor i of vertex:
        if color[i] == c: return false
    return true
```

Step 5: Chromatic Number Search

- upper = greedyUpperBound() = 10
- for m = 2 to upper: if backtrack(0, m) return m
- Result: Chromatic number $\chi = 10$

4. Why This Algorithm

Comparison of Approaches:

Approach	Complexity	Optimality	Feasibility
Brute Force	O(10^50)	Guaranteed	Impossible
Greedy Only	$O(n^2 \log n)$	Not guaranteed	Fast but suboptimal

Backtracking + Pruning	O(m^n) worst-case	Guaranteed	Practical (52 calls)

Why Backtracking with Pruning: It systematically explores the search space while isSafe() prunes invalid branches early. Only 52 recursive calls were needed for 50 courses.

Why Adjacency Matrix: Simple 2D array with O(1) edge lookup, required by the assignment, suitable for dense graphs.

Why Welsh-Powell Heuristic**:** Used only to find upper bound quickly, reducing search range from 50 to 10.

---

5. Implementation Details

Programming Language: C++17

File Structure:

- main.cpp (170 lines) - Entry point and interactive menu
- graph.h/cpp (120 lines) - Graph class with adjacency matrix `bool adj[100][100]`
- coloring.h/cpp (250 lines) - GraphColoring class with backtracking, isSafe, findChromaticNumber, greedyUpperBound
- dataset.h/cpp (200 lines) - Builds 50 courses across 6 departments with 228 conflicts
- display.h/cpp (150 lines) - Output formatting functions

Key Data Structures:

- `bool adj[100][100]` - Adjacency matrix
- `vector<int> color` - Color assignments (1-based)
- `string courseNames[100]` - Course names
- `long long callCount` - Recursive calls counter

Key Algorithms:

1. Backtracking (recursive)
2. Welsh-Powell greedy heuristic
3. Constraint propagation (isSafe)
4. Chromatic number search

6. Experimentation

```
-------------------------------------------------------------+
        GRAPH COLORING  -  EXAM SCHEDULING SYSTEM            |
-------------------------------------------------------------+
  Problem   : Schedule exams so no student has 2 at once     |
  Model     : Graph  (courses=vertices, conflicts=edges)     |
  Algorithm : Backtracking with Constraint Pruning           |
  Structure : Adjacency Matrix  (2-D boolean array)          |
  Dataset   : 50 University Courses, 228 Conflict Edges      |
  Goal      : Find minimum time slots = Chromatic Number     |
-------------------------------------------------------------+


=============================================================+
                        GRAPH SUMMARY                        |
=============================================================+

  What is this graph?
    Each VERTEX  = one university course
    Each EDGE    = two courses share at least one student
    Each COLOR   = one exam time slot

  Graph size:
    Vertices (courses)  : 50
    Edges (conflicts)   : 228

  Departments in dataset:
```

```
  Departments in dataset:
  Department                    Courses    Vertex Range
  ---------------------------------------------------
  Computer Science              10         1 - 10
  Mathematics                   10         11 - 20
  Physics                       8          21 - 28
  Electrical Eng.               9          29 - 37
  Business                      7          38 - 44
  Electives (cross-dept)        6          45 - 50

  Conflict rules:
    - All courses within a dept conflict (dense clique)
    - Cross-dept electives add realistic extra conflicts
-------------------------------------------------------------

-----------------------------------------+
                MAIN MENU                |
-----------------------------------------+
  [1]  Find Chromatic Number (auto)      |
  [2]  Color with custom m colors        |
  [3]  Show Adjacency Matrix             |
  [4]  Show Edge List (conflict pairs)   |
  [5]  Show Exam Schedule (by time slot) |
  [6]  Show Color Assignments (per course) |
  [7]  Verify Solution                   |
  [8]  Show Algorithm Statistics         |
  [0]  Exit                              |
-----------------------------------------+
Choice: 1
```

Test 1: Find Chromatic Number

```
Choice: 1

+==========================================================+
|                  FINDING CHROMATIC NUMBER                |
+==========================================================+

[Chromatic Search] Greedy upper bound = 10 colors
[Chromatic Search] Trying m = 2 ... fail  3 ... fail  4 ... fail  5 ... f
il  6 ... fail  7 ... fail  8 ... fail  9 ... fail  10 ... SUCCESS

+-----------------------------------------+
|  RESULT: Chromatic Number X = 10         |
|  Meaning: Need exactly 10 exam time slots.  |
|  No student will have 2 exams at once.   |
+-----------------------------------------+
```

- m = 2 through 9 → FAILED, m = 10 → SUCCESS
- Result: χ = 10, recursive calls = 52

Test 2: Custom m = 3 → FAILED (CS clique requires 10 colors)

```
Choice: 2
Enter number of colors (time slots) m: 10

+==============================================================+
|                      COLORING WITH m = 10                    |
+==============================================================+

[SUCCESS] Valid coloring found with 10 colors.
Use option [5] to see the schedule.
```

```
Choice: 5

+=========================================================+
|              FINAL EXAM SCHEDULE                        |
|   (Each slot = one exam period, no student conflict)    |
+=========================================================+

+-- TIME SLOT 1  (11 exam(s)) ----------------------------------+
|   [1 ]  CS101-Intro_Programming                            |
|   [11]  MATH101-Calculus_I                                 |
|   [22]  PHY102-Electromagnetism                            |
|   [30]  EE201-Electronics                                  |
|   [38]  BUS101-Accounting                                  |
|   [45]  ELEC-Data_Science                                  |
|   [46]  ELEC-Robotics                                      |
|   [47]  ELEC-Cybersecurity                                 |
|   [48]  ELEC-Bioinformatics                                |
|   [49]  ELEC-Game_Theory                                   |
|   [50]  ELEC-Quantum_Computing                             |

+-- TIME SLOT 2  (5 exam(s)) ----------------------------------+
|   [2 ]  CS201-Data_Structures                              |
|   [12]  MATH102-Calculus_II                                |
|   [21]  PHY101-Mechanics                                   |
|   [29]  EE101-Circuit_Theory                               |
```

```
- TIME SLOT 2  (5 exam(s)) ----------------------------------+
[2 ]  CS201-Data_Structures                                |
[12]  MATH102-Calculus_II                                  |
[21]  PHY101-Mechanics                                     |
[29]  EE101-Circuit_Theory                                 |
[39]  BUS102-Economics                                     |

- TIME SLOT 3  (5 exam(s)) ----------------------------------+
[3 ]  CS301-Algorithms                                     |
[13]  MATH201-Linear_Algebra                               |
[24]  PHY202-Quantum_Mechanics                             |
[31]  EE202-Digital_Logic                                  |
[40]  BUS201-Finance                                       |

- TIME SLOT 4  (5 exam(s)) ----------------------------------+
[4 ]  CS302-OS                                             |
[14]  MATH202-Discrete_Math                                |
[23]  PHY201-Thermodynamics                                |
[32]  EE301-Signal_Processing                              |
[41]  BUS202-Marketing                                     |

- TIME SLOT 5  (5 exam(s)) ----------------------------------+
[5 ]  CS401-Compilers                                      |
[15]  MATH301-Probability                                  |
[25]  PHY301-Optics                                        |
[33]  EE302-Control_Systems                                |
[42]  BUS301-Management                                    |

- TIME SLOT 6  (5 exam(s)) ----------------------------------+
[6 ]  CS402-Networks                                       |
```

Test 3: Custom m = 9 → FAILED (confirms 10 is minimum)

Test 4: Custom m = 10 → SUCCESS

Test 5: Custom m = 12 → SUCCESS (valid but not optimal)

Test 6: Exam Schedule (Selected Results)

```
+-------------------------------------------------------------+
|  COLOR ASSIGNMENTS (per course)
    |
+-------------------------------------------------------------+
| No.    | Course Name                        | Time Slot     | Color    |
+-------------------------------------------------------------+
| 1      | CS101-Intro_Programming            | Slot-1        | Color-1  |
| 2      | CS201-Data_Structures              | Slot-2        | Color-2  |
| 3      | CS301-Algorithms                   | Slot-3        | Color-3  |
| 4      | CS302-OS                           | Slot-4        | Color-4  |
| 5      | CS401-Compilers                    | Slot-5        | Color-5  |
| 6      | CS402-Networks                     | Slot-6        | Color-6  |
| 7      | CS403-AI                           | Slot-7        | Color-7  |
| 8      | CS404-ML                           | Slot-8        | Color-8  |
| 9      | CS405-Database                     | Slot-9        | Color-9  |
| 10     | CS406-Security                     | Slot-10       | Color-10|
| 11     | MATH101-Calculus_I                 | Slot-1        | Color-1  |
| 12     | MATH102-Calculus_II                | Slot-2        | Color-2  |
| 13     | MATH201-Linear_Algebra             | Slot-3        | Color-3  |
| 14     | MATH202-Discrete_Math              | Slot-4        | Color-4  |
| 15     | MATH301-Probability                | Slot-5        | Color-5  |
| 16     | MATH302-Statistics                 | Slot-6        | Color-6  |
| 17     | MATH401-Numerical_Methods          | Slot-7        | Color-7  |
| 18     | MATH402-Graph_Theory               | Slot-8        | Color-8  |
| 19     | MATH403-Optimization               | Slot-10       | Color-10|
| 20     | MATH404-Cryptography               | Slot-9        | Color-9  |
| 21     | PHY101-Mechanics                   | Slot-2        | Color-2  |
| 22     | PHY102-Electromagnetism            | Slot-1        | Color-1  |
| 23     | PHY201-Thermodynamics              | Slot-4        | Color-4  |
| 24     | PHY202-Quantum_Mechanics           | Slot-3        | Color-3  |
```

As a samples but 50.

Test 7: Solution Verification → All 228 edges checked, zero violations

```
Choice: 7

+=============================================================+
|                    SOLUTION VERIFICATION                    |
+=============================================================+

  Checking all 228 conflict edges...
  Rule: if edge(A,B) exists then color[A] != color[B]

  [PASS] All 50 courses assigned a slot.
  [PASS] All 228 conflict edges checked - zero violations.
  [PASS] Solution is VALID and OPTIMAL.
+-------------------------------------------------------------+
```

Test 8: Algorithm Statistics

```
+-----------------------------------------------------+
|            ALGORITHM STATISTICS                     |
+-----------------------------------------------------+
|  Graph                                              |
|    Vertices (courses)    : 50                       |
|    Edges (conflicts)     : 228                      |
+-----------------------------------------------------+
|  Result                                             |
|    Chromatic number (X) : 10                        |
|    Meaning              : 10 time slots needed      |
+-----------------------------------------------------+
|  Backtracking Performance                           |
|    Recursive calls made : 52                        |
|    Worst-case (theory)  : O(m^n)                    |
|    Actual O(m^n) value  : O(10^50)                    |
|    Pruning saved calls  : enormous (see above)    |
+-----------------------------------------------------+
|  Real-World Meaning                                 |
|    Only 10 exam periods needed for 50 courses.        |
|    No student has 2 exams at the same time.       |
```

- Vertices: 50, Edges: 228, Colors: 10
- Recursive calls: 52 (vs theoretical 10^50)
- Pruning efficiency: ~10^48 reduction factor

7. Learned Challenges

Challenge 1: Understanding NP-Hardness

- ✓ Problem: Didn't understand why greedy fails
- ✓ Learning: Greedy gives approximations, not optimal solutions
- ✓ Takeaway: Trade-off between speed and correctness

Challenge 2: Building a Realistic Dataset

- ✓ Problem: Creating realistic conflicts was complex
- ✓ Learning: Research university department structures
- ✓ Solution: Intra-department cliques + cross-department edges

Challenge 3: Implementing Adjacency Matrix

- ✓ Problem: Matrix must be symmetric for undirected graphs
- ✓ Solution: addEdge(u,v) sets both adj[u][v] and adj[v][u]

- ✓ Challenge 4: Backtracking Logic

- ✓ Problem: Forgetting to reset color[vertex]=0 on backtrack
- ✓ Solution: Always undo state changes when backtracking

Challenge 5: Finding Chromatic Number Efficiently

- ✓ Problem: Starting from m=1 is very slow
- ✓ Solution: Use Welsh-Powell upper bound to limit search

- ✓ Challenge 6: Understanding Pruning Efficiency

- ✓ Problem: Didn't understand why 52 calls vs 10^50
- ✓ Learning: is Safe () eliminates entire branches early

8. Conclusion

The study demonstrates that the backtracking algorithm successfully determined the chromatic number $\chi = 10$, indicating that Aksum University requires exactly 10 time slots to schedule 50 courses without conflicts. The solution was thoroughly verified against all 228 conflict edges, with zero violations, confirming that it is both valid and optimal. In terms of performance, the use of pruning techniques significantly reduced the worst-case search space from 10^{50} possibilities to only 52 recursive calls, representing an impressive reduction factor of approximately 10^{48} and highlighting the efficiency of backtracking with constraint pruning.

Throughout the project, several key achievements were realized, including the successful implementation of backtracking with m colors, the use of an adjacency matrix (2D array) to represent the graph, and the creation of a dataset containing more than 50 vertices. The chromatic number was accurately computed, and appropriate color assignments were displayed, while the algorithm's $O(m^n)$ complexity was analyzed and linked to a real-world application. The system also demonstrated strong relevance to practical scenarios such as university exam timetabling, compiler register allocation, radio frequency assignment, geographic map coloring, and parallel task scheduling.

For future improvements, the system can be enhanced by implementing more advanced algorithms such as DSATUR and genetic algorithms, developing a web-based interface, and extending the model to support soft constraints and multi-day scheduling. Overall, the project provided valuable learning outcomes, including a deeper understanding of NP-hard problems, hands-on experience with backtracking algorithms, practical application of graph theory to real-world problems, improved C++ programming skills, and insight into algorithm complexity analysis

---

5.

Compilation:

```text
g++ -std=c++17 -O2 src/*.cpp -o exam_scheduler.exe
```

Execution:

```text
./exam_scheduler.exe
```

Complexity Analysis:

- Time Complexity (Worst case): O(m^n) = O(10^50)
- Time Complexity (Actual): 52 recursive calls due to pruning
- Space Complexity: $O(n^2)$ = O(2500)